

TokoJoko

Backend REST API E-Commerce (Toko Bunga)

Laravel 10 + Sanctum + MySQL

Mata Kuliah

Pemrograman Sisi Server (PSS)

Nama Mahasiswa

NIM

Kelas

Repository

https://github.com/imdevedugame/e-commerce_new

Tanggal

14 Juli 2026

Daftar Isi

1. Latar Belakang
2. Analisa dan Rancangan Aplikasi, Pemodelan UML
 1. Analisa Kebutuhan
 2. Analisa Kebutuhan Hardware dan Software
 3. Skema Diagram Relasi Tabel
 4. Model
 5. Query
 6. Import CSV
 7. API
 8. Auth
 9. Throttling, Pagination, Filtering, Sorting
 10. Docs
 11. Unit Testing
3. Implementasi Sistem
4. Penutup
5. Lampiran Code Program

1 Latar Belakang

Bisnis ritel bunga (florist) semakin banyak berpindah dari transaksi tatap muka ke penjualan daring. Aplikasi **TokoJoko** yang telah ada sebelumnya melayani hal ini lewat storefront berbasis Livewire (server-rendered, sesi/cookie) yang hanya bisa diakses lewat browser di alamat web tertentu. Kebutuhan pasar saat ini menuntut lebih dari itu: penjual ingin produknya juga bisa diakses lewat **aplikasi mobile** (Android/iOS) atau diintegrasikan ke **aplikasi pihak ketiga** lain, sesuatu yang tidak mungkin dilakukan oleh arsitektur berbasis sesi/cookie karena client eksternal tidak berbagi cookie browser dengan server.

Untuk itu, backend TokoJoko dikembangkan lebih lanjut dengan menambahkan **layer REST API** (`/api/v1/*`) yang bersifat stateless dan berbasis token, sehingga bisa dikonsumsi oleh client manapun — baik untuk keperluan tugas mata kuliah Pemrograman Sisi Server (PSS) ini, maupun untuk disambungkan ke aplikasi client lain seperti tugas Pemrograman Perangkat Bergerak (PPB) atau Pemrograman Sisi Client (PSC).

Laporan ini mendokumentasikan rancangan, implementasi, dan pengujian backend tersebut, mencakup desain database, pemodelan UML, implementasi API lengkap dengan autentikasi, pembatasan request (throttling), pagination, filtering, sorting, fitur import data massal (CSV), pengujian otomatis (unit/feature test), dokumentasi API, hingga kesiapan hosting.

2 Analisa dan Rancangan Aplikasi, Pemodelan UML

Bagian ini menjelaskan analisa kebutuhan, rancangan basis data, dan pemodelan UML (use case diagram, class diagram, sequence diagram) sebagai dasar implementasi, dilanjutkan dengan pembahasan teknis tiap komponen backend (Model, Query, Import CSV, API, Auth, Throttling/ Pagination/Filtering/Sorting, Docs, dan Unit Testing).

2.1 Analisa Kebutuhan

Kebutuhan Fungsional

#	Kebutuhan
F1	Pengguna dapat mendaftar (register) dan masuk (login) untuk mendapatkan token akses API.
F2	Pengguna (guest maupun terautentikasi) dapat melihat daftar & detail produk beserta kategori, dengan pencarian, filter harga, dan pengurutan.
F3	Pengguna terautentikasi dapat membuat order (checkout) dengan validasi stok otomatis.
F4	Pengguna hanya dapat melihat order miliknya sendiri; admin dapat melihat seluruh order.
F5	Admin dapat mengimpor banyak produk sekaligus lewat file CSV.
F6	Pengguna dapat logout, mencabut token yang sedang aktif.
F7	Sistem menyediakan dokumentasi API yang bisa dicoba langsung (Swagger UI) dan diimpor ke Postman.

Kebutuhan Non-Fungsional

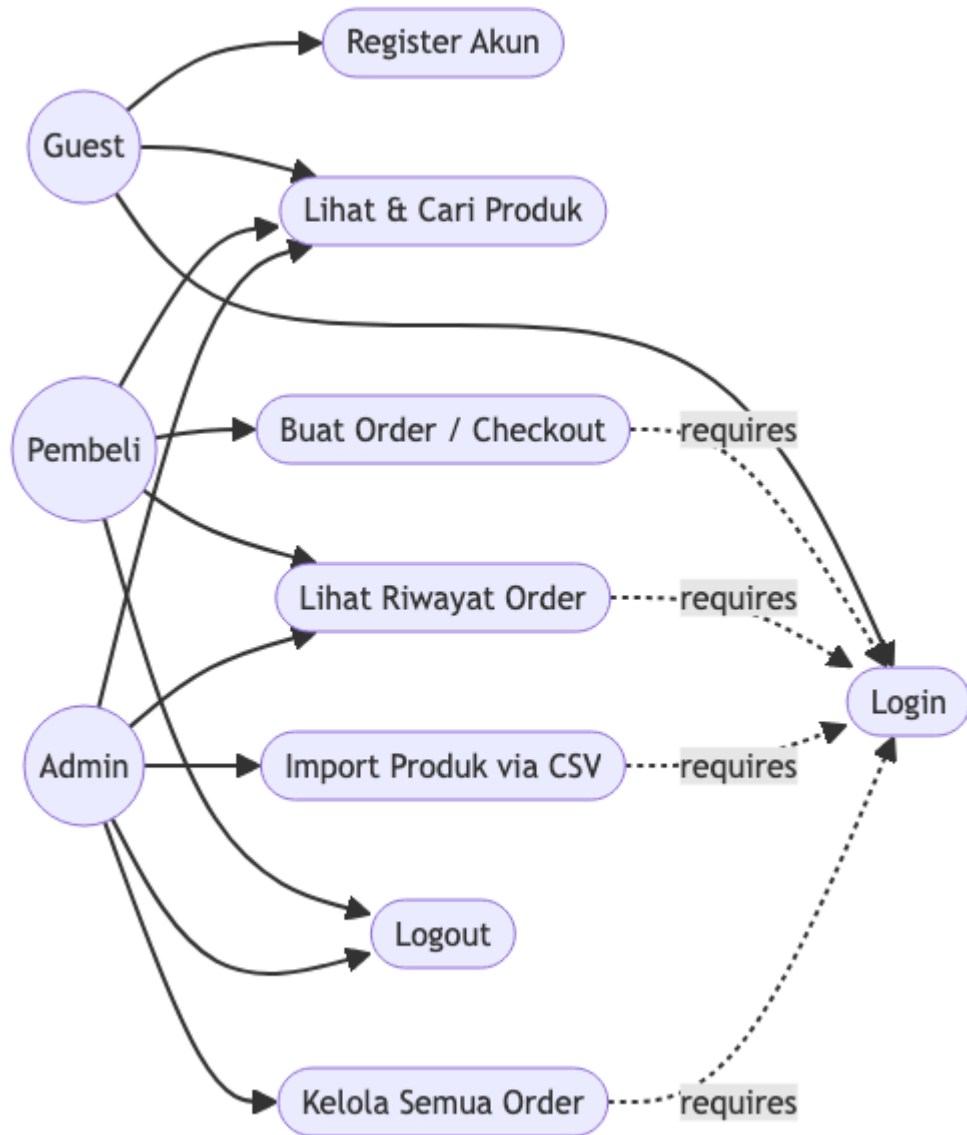
#	Kebutuhan
N1	Keamanan: autentikasi berbasis token (Sanctum), otorisasi kepemilikan resource (policy), validasi input di setiap endpoint.
N2	Ketersediaan & anti-abuse: seluruh endpoint dibatasi rate limit (throttling) untuk mencegah penyalahgunaan/brute-force.
N3	Skalabilitas: API bersifat stateless (token, bukan session) sehingga dapat di-deploy dan diskalakan terpisah dari storefront.
N4	Kualitas & keandalan kode: dicakup automated test (unit/feature) sehingga regresi dapat terdeteksi dini.
N5	Kemudahan integrasi: didokumentasikan lengkap (OpenAPI/Swagger, Postman, Markdown) agar mudah dikonsumsi client lain (mobile/SPA).

2.2 Analisa Kebutuhan Hardware dan Software

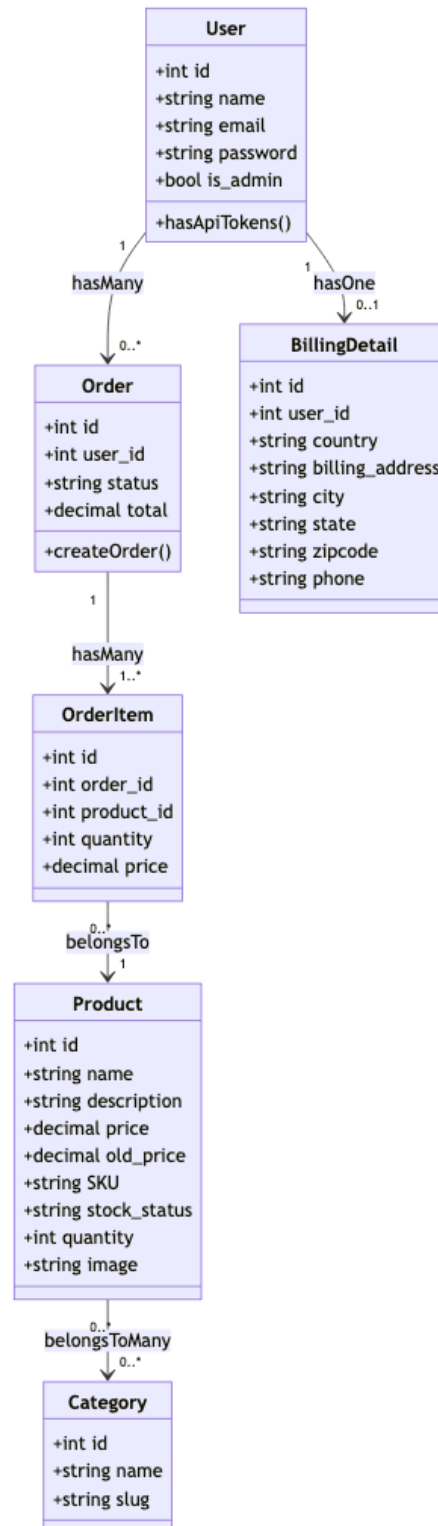
Kategori	Kebutuhan Minimum
Server (hosting)	1 vCPU, RAM 1 GB, storage 5 GB — cukup untuk PHP-FPM + MySQL skala kecil-menengah (mis. VPS/shared hosting Railway, Render, Niagahoster)
Sistem Operasi Server	Linux (Ubuntu 22.04 atau setara), mendukung PHP 8.1+
Software Server	PHP 8.1+ (diuji di 8.4), Composer 2, MySQL 8 / MariaDB 10.6+, Nginx/Apache, ekstensi PHP: mbstring, pdo_mysql, openssl, tokenizer, xml, ctype, json
Development	PHP 8.4, Composer, Node.js + npm (asset build Livewire/Filament), MySQL 8 lokal, Git
Client (konsumen API)	Perangkat apapun dengan koneksi internet & HTTP client — browser (Swagger UI), Postman, atau aplikasi mobile (Android/iOS) yang bisa memanggil REST API & menyimpan bearer token

2.3 Pemodelan UML

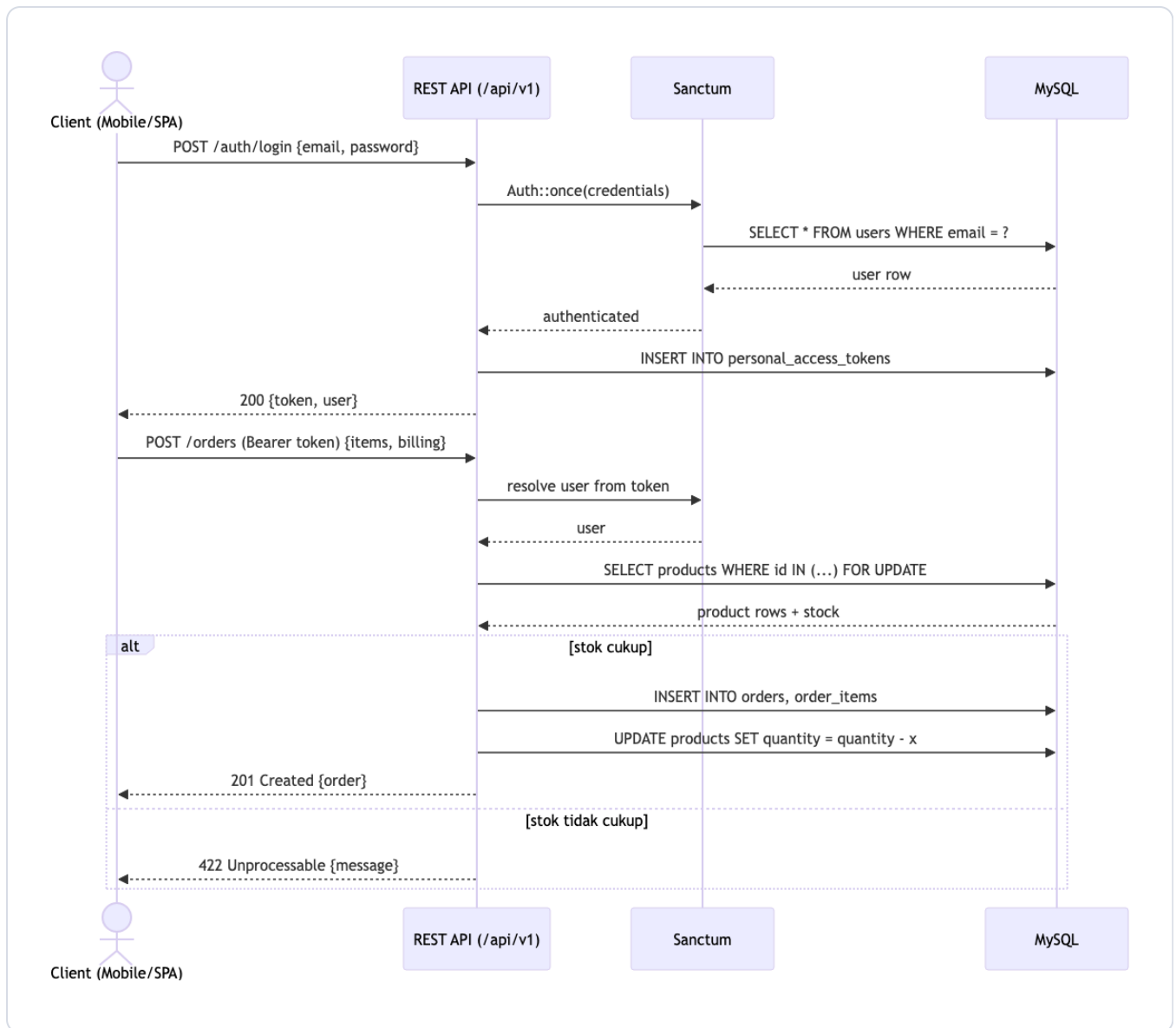
Use Case Diagram — aktor *Guest*, *Pembeli* (user terautentikasi), dan *Admin*, beserta fungsi yang dapat mereka akses:



Class Diagram — struktur model Eloquent beserta atribut, relasi, dan multiplisitasnya:

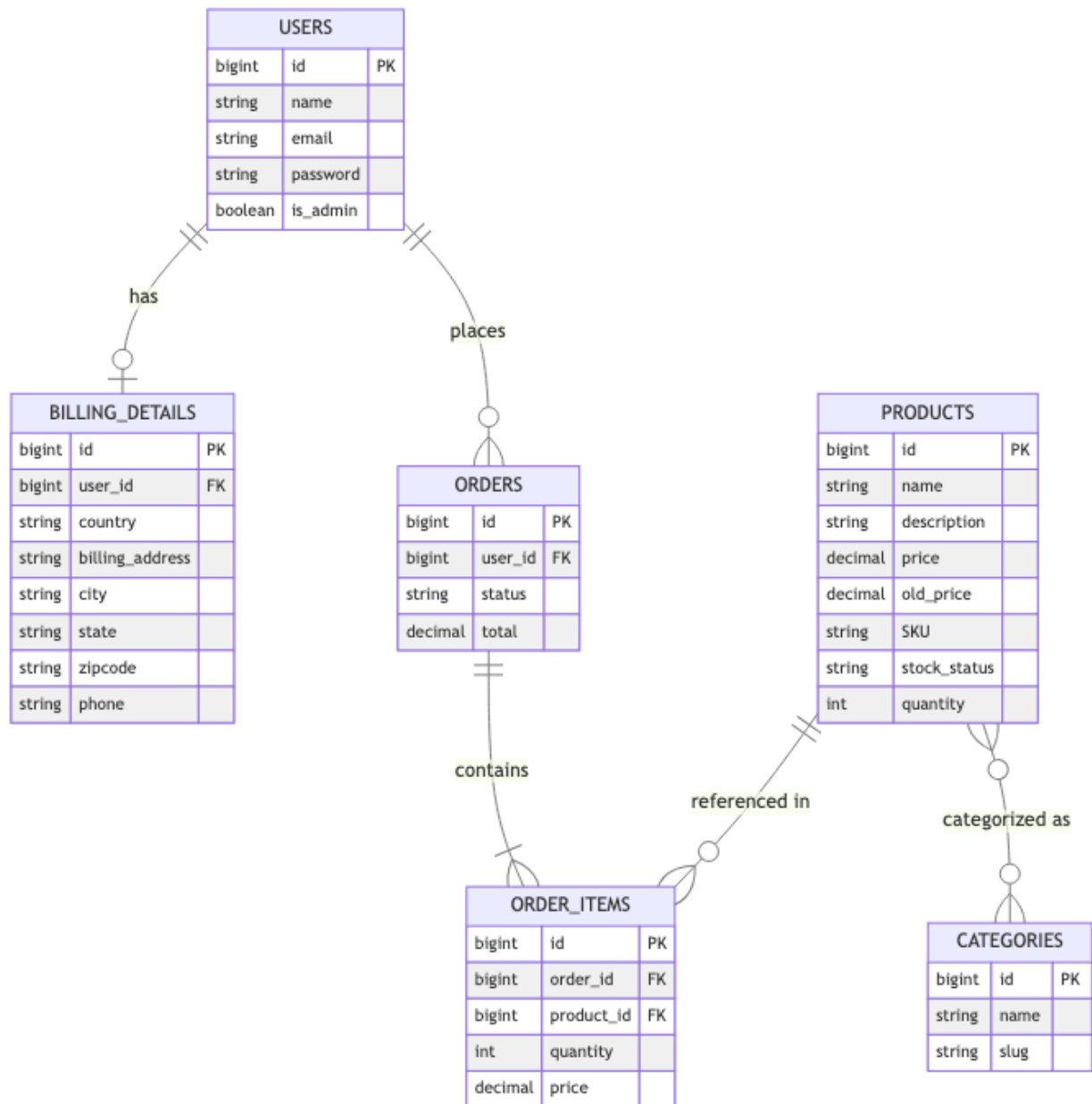


Sequence Diagram — alur login lalu checkout (order), termasuk percabangan saat stok tidak cukup:



2.4 Skema Diagram Relasi Tabel

Skema database terdiri dari 6 tabel inti, dikelola lewat Eloquent ORM dan migration Laravel ([database/migrations/](#)).



Tabel	Kolom Utama	Relasi
users	name, email, password, is_admin	hasMany Order, hasOne BillingDetail
categories	name, slug	belongsToMany Product (pivot category_product)
products	name, description, price, old_price, SKU, stock_status, quantity, image, images	belongsToMany Category
orders	user_id, status, total, session_id	belongsTo User, hasMany OrderItem
order_items	order_id, product_id, quantity, price	belongsTo Order, belongsTo Product
billing_details	user_id, country, billing_address, city, state, zipcode, phone, order_notes	belongsTo User

2.5 Model

Contoh migration tabel `orders` :

```
Schema::create('orders', function (Blueprint $table) {
    $table->id();
    $table->unsignedBigInteger('user_id');
    $table->enum('status', ['pending', 'processing', 'completed', 'canceled']);
    $table->decimal('total', 8, 2);
    $table->string('session_id')->nullable();
    $table->timestamps();
    $table->foreign('user_id')->references('id')->on('users')->onDelete('cascade');
});
```

Contoh model Eloquent `Order` (relasi & mass-assignment protection):

```
class Order extends Model
{
    use HasFactory;
    protected $fillable = ['user_id', 'status', 'total', 'session_id'];

    public function user()      { return $this->belongsTo(User::class); }
    public function orderItems(){ return $this->hasMany(OrderItem::class); }
}
```

Setiap model menggunakan `$fillable` untuk mass-assignment protection, dan foreign key di migration memakai `onDelete('cascade')` agar data anak (order_items, category_product) ikut terhapus saat parent-nya dihapus — menjaga integritas referensial database.

2.6 Query

Query dibangun lewat Eloquent Query Builder (bukan raw SQL), yang otomatis melakukan parameter binding — mencegah SQL Injection. Berikut query nyata yang dihasilkan (ditangkap lewat `DB::enableQueryLog()`) untuk endpoint `GET /products?category=mawar&min_price=50000&sort=price_asc` :

SQL ASLI YANG DIEKSEKUSI MYSQL (QUERY LOG)

```
select count(*) as aggregate from `products`
where exists (select * from `categories`
  inner join `category_product` on `categories`.`id` = `category_product`.`category_id`
  where `products`.`id` = `category_product`.`product_id` and `categories`.`slug` = ?)
and `price` >= ?
-- bindings: ["mawar", 50000]

select * from `products`
where exists (select * from `categories`
  inner join `category_product` on `categories`.`id` = `category_product`.`category_id`
  where `products`.`id` = `category_product`.`product_id` and `categories`.`slug` = ?)
and `price` >= ?
order by `price` asc limit 5 offset 0
-- bindings: ["mawar", 50000]

select `categories`.*, `category_product`.`product_id`, `category_product`.`category_id`
from `categories` inner join `category_product` on `categories`.`id` =
`category_product`.`category_id`
where `category_product`.`product_id` in (1, 8)
-- bindings: []
```

Query ke-3 adalah eager-load relasi `categories` (dipanggil otomatis oleh `with('categories')`) — teknik ini mencegah masalah N+1 query dibanding memuat kategori satu per satu untuk tiap produk.

Untuk endpoint `GET /orders?status=pending` (daftar order milik user):

SQL ASLI YANG DIEKSEKUSI MYSQL (QUERY LOG)

```
select count(*) as aggregate from `orders` where `user_id` = ? and `status` = ?
-- bindings: [1, "pending"]
```

2.7 Import CSV

Untuk kebutuhan admin yang ingin menambah banyak produk sekaligus (misalnya migrasi data dari toko lama atau supplier), disediakan endpoint `POST /api/v1/products/import` (khusus admin) yang menerima file CSV dan melakukan *upsert* berdasarkan SKU.

Format CSV

Kolom wajib: `name`, `sku`, `price`, `quantity`. Kolom opsional: `brief_description`, `description`, `old_price`, `stock_status`, `image`, `categories` (slug kategori dipisah koma). Contoh (`docs/products_import_sample.csv`):

```
name,brief_description,description,price,old_price,sku,stock_status,quantity,image,categories
Buket Anyelir Pink,Anyelir pink lembut,Rangkaian anyelir segar,95000,120000,ANY-001,instock,15,img.jpg,mawar
Buket Krisan Putih,Krisan putih elegan,Cocok acara formal,110000,140000,KRS-001,instock,10,img2.jpg,"lily,tulip"
```

Cuplikan Kode — ProductImportController

```
while (($row = fgetcsv($handle)) !== false) {
    $data = array_combine($header, array_pad($row, count($header), null));
    $validator = Validator::make($data, [
        'name' => ['required', 'string', 'max:255'],
        'sku' => ['required', 'string', 'max:255'],
        'price' => ['required', 'numeric', 'min:0'],
        'quantity' => ['required', 'integer', 'min:0'],
        // ...kolom opsional lain
    ]);

    if ($validator->fails()) {
        $errors[] = ['row' => $rowNumber, 'errors' => $validator->errors()->all()];
        continue; // baris invalid dilewati, file tetap lanjut diproses
    }

    $existed = Product::where('SKU', $valid['sku'])->exists();
    $product = Product::updateOrCreate(['SKU' => $valid['sku']], [ /* ...kolom lain */ ]);
    $existed ? $updated++ : $imported++;

    if (! empty($valid['categories'])) {
        $categoryIds = Category::whereIn('slug', explode(',', $valid['categories']))->pluck('id');
        $product->categories()->syncWithoutDetaching($categoryIds);
    }
}
```

Bukti Nyata — Import Dijalankan ke Server Sungguhan

CURL -F "FILE=@DOCS/PRODUCTS_IMPORT_SAMPLE.CSV" (ADMIN TOKEN)

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{"message":"Import completed.","summary":{"created":2,"updated":0,"failed":0},"errors":[]}
```

2 produk baru (ANY-001, KRS-001) berhasil dibuat di database MySQL sungguhan pada percobaan ini, lalu dibersihkan kembali setelah pengujian selesai. Endpoint ini juga dilindungi middleware `admin.api` — non-admin mendapat `403`, tanpa token mendapat `401` (lihat Bagian 2.11 Unit Testing).

2.8 API

Seluruh endpoint API berada di bawah prefix `/api/v1`, didaftarkan di `routes/api.php`, dan diproses oleh controller khusus di namespace `App\Http\Controllers\Api`.

Method	Endpoint	Auth	Keterangan
POST	/api/v1/auth/register	—	Registrasi user baru, langsung menerima token
POST	/api/v1/auth/login	—	Login, menerima bearer token
POST	/api/v1/auth/logout	✓	Mencabut token yang sedang dipakai
GET	/api/v1/auth/me	✓	Profil user yang sedang login
GET	/api/v1/categories	—	List kategori (pagination)
GET	/api/v1/categories/{id}	—	Detail kategori
GET	/api/v1/products	—	List produk (pagination, filter, search, sort)
GET	/api/v1/products/{id}	—	Detail produk
POST	/api/v1/products/import	✓ (admin)	Import massal produk dari CSV
GET	/api/v1/orders	✓	List order milik user (admin melihat semua)
POST	/api/v1/orders	✓	Membuat order (checkout) + validasi stok
GET	/api/v1/orders/{id}	✓	Detail order (hanya pemilik/admin)

Definisi Routing (routes/api.php)

```

Route::prefix('v1')->group(function () {
    Route::post('/auth/register', [AuthController::class, 'register'])->middleware('throttle:6,1');
    Route::post('/auth/login', [AuthController::class, 'login'])->middleware('throttle:6,1');

    Route::middleware('auth:sanctum')->group(function () {
        Route::post('/auth/logout', [AuthController::class, 'logout']);
        Route::get('/auth/me', [AuthController::class, 'me']);
    });

    Route::get('/categories', [CategoryController::class, 'index']);
    Route::get('/categories/{category}', [CategoryController::class, 'show']);
    Route::get('/products', [ProductController::class, 'index']);
    Route::get('/products/{product}', [ProductController::class, 'show']);

    Route::middleware(['auth:sanctum', 'admin.api'])->group(function () {
        Route::post('/products/import', [ProductImportController::class, 'import']);
    });

    Route::middleware('auth:sanctum')->group(function () {
        Route::get('/orders', [OrderController::class, 'index']);
        Route::post('/orders', [OrderController::class, 'store']);
        Route::get('/orders/{order}', [OrderController::class, 'show']);
    });
});

```

Bukti Nyata — Membuat Order (Checkout)

HTTP/1.1 201 CREATED

```
POST /api/v1/orders
Authorization: Bearer <token>
Content-Type: application/json

{ "items": [ { "product_id": 7, "quantity": 2 } ],
  "billing": { "country": "Indonesia", "billing_address": "Jl. Contoh No.1",
               "city": "Bandung", "state": "Jawa Barat", "zipcode": "40123", "phone": "081234567890" }
}

→ {"message":"Order created successfully.,"data":{"id":5,"status":"pending","total":50000,
    "items":[{"id":5,"product_id":7,"product_name":"ZZZ Demo Product Laporan","quantity":2,
    "price":25000,"subtotal":50000}], ...}}
```

2.9 Auth

Autentikasi API menggunakan **Laravel Sanctum** dengan skema *Personal Access Token* (bukan cookie/session seperti storefront Livewire), sehingga API benar-benar stateless dan cocok dikonsumsi aplikasi mobile maupun SPA eksternal.

- `POST /auth/register` dan `POST /auth/login` mengembalikan token dalam bentuk Bearer string.
- Setiap endpoint privat diproteksi middleware `auth:sanctum` dan mengharuskan header `Authorization: Bearer <token>`.
- Endpoint admin (`/products/import`) tambahan diproteksi middleware `admin.api` (`app/Http/Middleware/EnsureApiUserIsAdmin.php`).
- Tanpa token → `401 Unauthenticated`. Token valid tapi mengakses resource orang lain → `403 Forbidden` (lihat `app/Policies/OrderPolicy.php`).

```
class OrderPolicy
{
    public function view(User $user, Order $order): bool
    {
        return $user->is_admin || $user->id === $order->user_id;
    }
}
```

GET /AUTH/ME — TANPA VS DENGAN TOKEN

```
TANPA token → HTTP/1.1 401 Unauthorized {"message":"Unauthenticated."}
DENGAN token → HTTP/1.1 200 OK {"user":{"id":6,"name":"Demo User","email":"...","is_admin":false}}
```

2.10 Throttling, Pagination, Filtering, Sorting

Throttling

Cakupan	Batas	Response saat terlampaui
Seluruh <code>/api/*</code>	60 request/menit per user (atau per IP untuk tamu)	429 Too Many Requests
<code>/auth/register</code> , <code>/auth/login</code>	6 request/menit per IP (tambahan, lebih ketat)	429 Too Many Requests

```
// app/Providers/RouteServiceProvider.php
RateLimiter::for('api', function (Request $request) {
    return Limit::perMinute(60)->by($request->user()?->id ?: $request->ip());
});

// routes/api.php
Route::post('/auth/register', [...])->middleware('throttle:6,1');
Route::post('/auth/login', [...])->middleware('throttle:6,1');
```

8X CURL -I POST /AUTH/LOGIN (KREDENSIAL SALAH, BERUNTUN) — BUKTI NYATA 429

```
Percobaan ke-1..4 -> HTTP/1.1 422 | X-RateLimit-Limit: 6 | "The provided credentials are incorrect."
Percobaan ke-5..8 -> HTTP/1.1 429 | X-RateLimit-Limit: 60 | "Too Many Attempts."
```

Setelah 4 percobaan gagal, limiter 6/menit milik `/auth/login` habis sehingga percobaan ke-5 dst langsung diblokir 429 — dibuktikan langsung ke server yang berjalan, bukan cuma klaim di dokumen.

Pagination

```
{ "data": [ ... ], "meta": { "current_page": 1, "last_page": 3, "per_page": 12, "total": 30 } }
```

Parameter `per_page` dibatasi maksimum 50 (`max(1, min($perPage, 50))`) di controller.

Filtering & Sorting (khusus /products)

Parameter	Contoh	Efek
<code>category</code>	<code>?category=mawar</code>	Filter berdasarkan slug kategori
<code>search</code>	<code>?search=rose</code>	Cari di kolom name & description
<code>min_price</code> / <code>max_price</code>	<code>?min_price=10&max_price=100</code>	Filter rentang harga
<code>sort</code>	<code>?sort=price_asc</code>	Sorting: <code>latest</code> / <code>price_asc</code> / <code>price_desc</code>

2.11 Docs

Tersedia tiga bentuk dokumentasi API, saling melengkapi:

1. **Markdown reference** — `docs/API.md`
2. **Swagger UI interaktif** — `/docs` , dari spesifikasi OpenAPI 3.0 `public/openapi.json` (kini mencakup endpoint `/products/import`)

3. **Postman Collection** — `docs/postman_collection.json` , auto-simpan token, termasuk request import CSV

Unit Testing

Automated testing memakai PHPUnit, dijalankan di atas **SQLite in-memory** (`phpunit.xml`) — tidak pernah menyentuh database development (MySQL).

File Test	Cakupan
<code>AuthApiTest.php</code>	Register, login, duplikat email, salah password, rate limit, me, logout (8 test)
<code>ProductApiTest.php</code>	Pagination, filter kategori, search, filter harga, sorting, detail, 404 (7 test)
<code>CategoryApiTest.php</code>	List, detail, 404 (3 test)
<code>OrderApiTest.php</code>	Guest ditolak, checkout sukses+stok berkurang, stok kurang→422, isolasi per-user, 403, admin lihat semua (6 test)
<code>ProductImportApiTest.php</code>	Guest 401, non-admin 403, admin import+kategori, upsert by SKU, baris invalid dilewati, header CSV kurang→422 (6 test)

```
php artisan test          # seluruh test suite (54 test)
php artisan test --filter=Api # 30 test khusus API
```

\$ PHP ARTISAN TEST (DIJALANKAN ULANG SAAT LAPORAN INI DIBUAT)

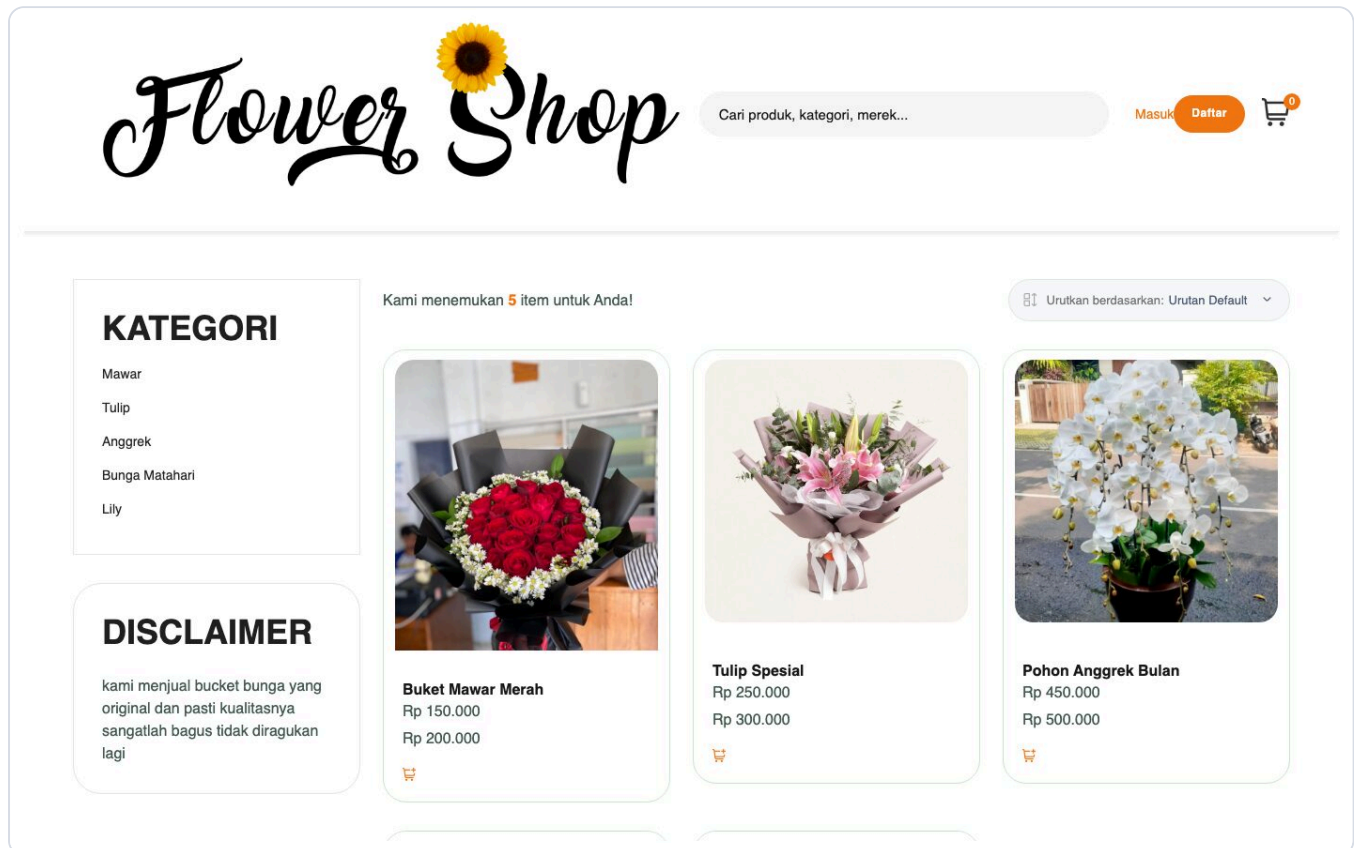
```
PASS Tests\Unit\ExampleTest (1 test)
PASS Tests\Feature\Api\AuthApiTest (8 test)
PASS Tests\Feature\Api\CategoryApiTest (3 test)
PASS Tests\Feature\Api\OrderApiTest (6 test)
PASS Tests\Feature\Api\ProductApiTest (7 test)
PASS Tests\Feature\Api\ProductImportApiTest (6 test)
PASS Tests\Feature\Auth\* (bawaan Breeze, 15 test)
PASS Tests\Feature\ExampleTest (1 test)
PASS Tests\Feature\ProfileTest (5 test)

Tests: 54 passed (141 assertions)
Duration: 3.65s
```


3 Implementasi Sistem

Screenshot berikut diambil langsung dari server yang benar-benar dijalankan (`php artisan serve`) di atas database MySQL development, membuktikan sistem berjalan nyata (bukan mockup).

3.1 Storefront (Livewire) — Halaman Utama



Halaman utama menampilkan katalog produk (nama, harga, kategori) yang datanya sama persis dengan yang disajikan lewat REST API `/api/v1/products`.

3.2 Swagger UI — Dokumentasi API Interaktif

TokoJoko API

1.0.0 OAS 3.0

<http://127.0.0.1:8127/openapi.json>

REST API for the TokoJoko e-commerce backend (auth, categories, products, orders). Intended to be consumed by an external client (mobile app / SPA).

Servers

/api/v1 - Current host

Authorize

Auth

POST

/auth/register

Register a new user

POST

/auth/login

Log in and receive a bearer token

POST

/auth/logout

Revoke the current access token

GET

/auth/me

Get the authenticated user's profile

Categories

GET

/categories

List categories

GET

/categories/{id}

Show a single category

Products

GET

/products

List products

GET

/products/{id}

Show a single product

POST

/products/import

Bulk import products from a CSV file

Seluruh 12 endpoint tergroup per tag (Auth, Categories, Products, Orders), termasuk endpoint `/products/import` yang baru ditambahkan, lengkap dengan ikon gembok pada endpoint yang butuh autentikasi.

3.3 Menjalankan Server (untuk keperluan presentasi/demo)

```
composer install
cp .env.example .env
php artisan key:generate
# set DB_DATABASE / DB_USERNAME / DB_PASSWORD di .env
php artisan migrate
php artisan storage:link
php artisan serve
```

Server berjalan di `http://localhost:8000` ; dokumentasi interaktif di `http://localhost:8000/docs` .

4 Penutup

4.1 Kesimpulan

✓	Desain database, model & schema — 6 tabel, migration & model Eloquent lengkap dengan relasi (Bagian 2.4-2.5)
✓	Query — dibuktikan lewat SQL asli hasil query log, aman dari SQL Injection (parameter binding), eager loading mencegah N+1 (Bagian 2.6)
✓	Import CSV — endpoint admin, upsert by SKU, baris invalid dilewati tanpa gagalkan seluruh file, diuji nyata (Bagian 2.7)
✓	API — 12 endpoint REST stateless untuk auth, produk, kategori, order, import (Bagian 2.8)
✓	Auth — Sanctum bearer token + otorisasi kepemilikan resource (Bagian 2.9)
✓	Throttling, pagination, filtering, sorting — rate limit dua lapis (dibuktikan 429 nyata), pagination konsisten, filter & sort produk (Bagian 2.10)
✓	Docs — Markdown, Swagger UI, Postman Collection (Bagian 2.11)
✓	Unit testing — 54 automated test (30 khusus API), semua PASSED (Bagian 2.11)

Seluruh kebutuhan fungsional dan non-fungsional pada Bagian 2.1 telah dipenuhi dan diverifikasi baik lewat automated test maupun pengujian manual langsung ke server yang berjalan.

4.2 Saran Pengembangan Lanjutan

- Menambahkan refresh token / token expiry otomatis agar sesi API lebih aman untuk aplikasi mobile jangka panjang.
- Menyediakan endpoint upload gambar produk langsung (multipart) alih-alih hanya path string, agar admin tidak perlu mengelola file secara manual.
- Menambahkan halaman admin (Filament) untuk memicu import CSV lewat UI, bukan hanya lewat API/Postman.
- Menerapkan queue (antrian) untuk proses import CSV berukuran besar agar tidak memblokir request HTTP.
- Mempersempit `config/cors.php` ke origin klien resmi sebelum rilis produksi.

5 Lampiran Code Program

Kode lengkap tersedia di repository (https://github.com/imdedugame/e-commerce_new). Berikut cuplikan file-file inti backend REST API.

5.1 routes/api.php

```
<?php

use App\Http\Controllers\Api\AuthController;
use App\Http\Controllers\Api\CategoryController;
use App\Http\Controllers\Api\OrderController;
use App\Http\Controllers\Api\ProductController;
use App\Http\Controllers\Api\ProductImportController;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

Route::middleware('auth:sanctum')->get('/user', function (Request $request) {
    return $request->user();
});

Route::prefix('v1')->group(function () {
    Route::post('/auth/register', [AuthController::class, 'register'])->middleware('throttle:6,1');
    Route::post('/auth/login', [AuthController::class, 'login'])->middleware('throttle:6,1');

    Route::middleware('auth:sanctum')->group(function () {
        Route::post('/auth/logout', [AuthController::class, 'logout']);
        Route::get('/auth/me', [AuthController::class, 'me']);
    });

    Route::get('/categories', [CategoryController::class, 'index']);
    Route::get('/categories/{category}', [CategoryController::class, 'show']);
    Route::get('/products', [ProductController::class, 'index']);
    Route::get('/products/{product}', [ProductController::class, 'show']);

    Route::middleware(['auth:sanctum', 'admin.api'])->group(function () {
        Route::post('/products/import', [ProductImportController::class, 'import']);
    });

    Route::middleware('auth:sanctum')->group(function () {
        Route::get('/orders', [OrderController::class, 'index']);
        Route::post('/orders', [OrderController::class, 'store']);
        Route::get('/orders/{order}', [OrderController::class, 'show']);
    });
});
```

5.2 app/Http/Controllers/Api/ProductController.php

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Http\Resources\ProductResource;
use App\Models\Product;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    public function index(Request $request): JsonResponse
    {
        $query = Product::query()->with('categories');

        if ($category = $request->query('category')) {
            $query->whereHas('categories', function ($q) use ($category) {
                $q->where('categories.slug', $category);
            });
        }

        if ($search = $request->query('search')) {
            $query->where(function ($q) use ($search) {
                $q->where('name', 'like', "%{$search}%")
                    ->orWhere('description', 'like', "%{$search}%");
            });
        }

        if ($request->filled('min_price')) {
            $query->where('price', '>=', (float) $request->query('min_price'));
        }
        if ($request->filled('max_price')) {
            $query->where('price', '<=', (float) $request->query('max_price'));
        }

        match ($request->query('sort')) {
            'price_asc' => $query->orderBy('price', 'asc'),
            'price_desc' => $query->orderBy('price', 'desc'),
            'latest' => $query->orderBy('created_at', 'desc'),
            default => $query->orderBy('id', 'asc'),
        };

        $perPage = max(1, min((int) $request->query('per_page', 12), 50));
        $products = $query->paginate($perPage)->appends($request->query());

        return response()->json([
            'data' => ProductResource::collection($products->items()),
            'meta' => [
                'current_page' => $products->currentPage(),
                'last_page' => $products->lastPage(),
                'per_page' => $products->perPage(),
                'total' => $products->total(),
            ],
        ]);
    }

    public function show(Product $product): JsonResponse
    {
        $product->load('categories');
        return response()->json(['data' => new ProductResource($product)]);
    }
}
```

```

    }
}

```

5.3 app/Http/Controllers/Api/OrderController.php (store — checkout)

```

public function store(StoreOrderRequest $request): JsonResponse
{
    $order = DB::transaction(function () use ($request) {
        $user = $request->user();
        $items = $request->validated('items');

        $products = Product::query()->whereIn('id', array_column($items, 'product_id'))
            ->lockForUpdate()->get()->keyBy('id');

        $total = 0;
        $orderItemsData = [];
        foreach ($items as $item) {
            $product = $products->get($item['product_id']);
            if ($product->quantity < $item['quantity']) {
                throw ValidationException::withMessages([
                    'items' => ["Insufficient stock for product '{$product->name}'. Available: {$product->quantity}."],
                ]);
            }
            $subtotal = $product->price * $item['quantity'];
            $total += $subtotal;
            $orderItemsData[] = ['product' => $product, 'quantity' => $item['quantity'], 'price' => $product->price];
        }

        $order = Order::create(['user_id' => $user->id, 'status' => 'pending', 'total' => $total]);

        foreach ($orderItemsData as $data) {
            OrderItem::create([
                'order_id' => $order->id, 'product_id' => $data['product']->id,
                'quantity' => $data['quantity'], 'price' => $data['price'],
            ]);
            $data['product']->decrement('quantity', $data['quantity']);
            if ($data['product']->fresh()->quantity <= 0) {
                $data['product']->update(['stock_status' => 'outstock']);
            }
        }

        BillingDetail::updateOrCreate(['user_id' => $user->id], $request->validated('billing'));
        return $order;
    });

    $order->load('orderItems.product');
    return response()->json(['message' => 'Order created successfully.', 'data' => new OrderResource($order)], 201);
}

```

5.4 app/Http/Controllers/Api/ProductImportController.php

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Category;
use App\Models\Product;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Validator;

class ProductImportController extends Controller
{
    private const REQUIRED_COLUMNS = ['name', 'sku', 'price', 'quantity'];

    public function import(Request $request): JsonResponse
    {
        $request->validate(['file' => ['required', 'file', 'mimes:csv,txt']]);

        $handle = fopen($request->file('file')->getRealPath(), 'r');
        $header = array_map(fn ($col) => strtolower(trim($col)), fgetcsv($handle));

        $missing = array_diff(self::REQUIRED_COLUMNS, $header);
        if (! empty($missing)) {
            fclose($handle);
            return response()->json(['message' => 'CSV header is missing required column(s): '.implode(' ', $missing)], 422);
        }

        $imported = 0; $updated = 0; $errors = []; $rowNumber = 1;

        while (($row = fgetcsv($handle)) !== false) {
            $rowNumber++;
            if (count(array_filter($row, fn ($v) => $v !== null && $v !== '')) === 0) continue;

            $data = array_combine($header, array_pad($row, count($header), null));
            $validator = Validator::make($data, [
                'name' => ['required', 'string', 'max:255'],
                'sku' => ['required', 'string', 'max:255'],
                'price' => ['required', 'numeric', 'min:0'],
                'quantity' => ['required', 'integer', 'min:0'],
                'categories' => ['nullable', 'string'],
            ]);

            if ($validator->fails()) {
                $errors[] = ['row' => $rowNumber, 'errors' => $validator->errors()->all()];
                continue;
            }

            $valid = $validator->validated();
            DB::transaction(function () use ($valid, &$imported, &$updated) {
                $existed = Product::where('SKU', $valid['sku']->exists());
                $product = Product::updateOrCreate(['SKU' => $valid['sku']], [
                    'name' => $valid['name'], 'price' => $valid['price'], 'quantity' => $valid['quantity'],
                    'stock_status' => $valid['stock_status'] ?? 'instock', 'images' => json_encode([]),
                ]);
                $existed ? $updated++ : $imported++;
            });

            if (! empty($valid['categories'])) {
                $slugs = array_filter(array_map('trim', explode(',', $valid['categories'])));
            }
        }
    }
}
```

```

        $product->categories()->syncWithoutDetaching(Category::whereIn('slug', $slugs)-
>pluck('id'));
    }
    });
}
fclose($handle);

return response()->json([
    'message' => 'Import completed.',
    'summary' => ['created' => $imported, 'updated' => $updated, 'failed' => count($errors)],
    'errors' => $errors,
]);
}
}

```

5.5 app/Http/Middleware/EnsureApiUserIsAdmin.php

```

<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureApiUserIsAdmin
{
    public function handle(Request $request, Closure $next): Response
    {
        if (! $request->user() || ! $request->user()->is_admin) {
            abort(403, 'This action requires an administrator account.');
```


5.6 app/Http/Controllers/Api/AuthController.php

```
<?php

namespace App\Http\Controllers\Api;

class AuthController extends Controller
{
    public function register(RegisterRequest $request): JsonResponse
    {
        $user = User::create([
            'name' => $request->validated('name'),
            'email' => $request->validated('email'),
            'password' => Hash::make($request->validated('password')),
        ]);
        $token = $user->createToken('api-token')->plainTextToken;

        return response()->json([
            'message' => 'Registration successful.',
            'user' => new UserResource($user),
            'token' => $token,
        ], 201);
    }

    public function login(LoginRequest $request): JsonResponse
    {
        if (! Auth::once($request->validated())) {
            throw ValidationException::withMessages(['email' => ['The provided credentials are incorrect.']] );
        }
        $token = Auth::user()->createToken('api-token')->plainTextToken;
        return response()->json([
            'message' => 'Login successful.', 'user' => new UserResource(Auth::user()), 'token' => $token,
        ]);
    }

    public function logout(Request $request): JsonResponse
    {
        $request->user()->currentAccessToken()->delete();
        return response()->json(['message' => 'Logged out successfully.']);
    }

    public function me(Request $request): JsonResponse
    {
        return response()->json(['user' => new UserResource($request->user())]);
    }
}
```